

TEMA 3 – Debugear Apps

Objetivo del tema

En este tema aprenderás a utilizar las herramientas que ofrece Android Studio para depurar las Apps.

3.1 El debugger y el Logcat

Android Studio dispone de un debugger similar al de otros IDE a los que estás acostumbrado, simplemente es un poco más sofisticado. Al fin y al cabo, se conecta a una App móvil emulada en una máquina virtual... pero un programador no se dará cuenta de la diferencia.

En Android Studio es muy importante saber manejarse con el debugger porque:

- Es la única manera de encontrar bugs ocultos en el **ciclo de vida de Actividades** o en los **Fragments**
- Gestionar la ejecución de **hilos** es muy complejo.
- Existen ciertas interacciones propias con el sistema (rotaciones de móvil, pausas, etc.)

El **Logcat** de Android Studio es por su parte un sistema de registro que genera mensajes en tiempo real. Es algo muy similar al **Log4J** de Java, o incluso podría decirse que a poner `System.out.println` para ver qué anda haciendo tu programa.

Si bien es cierto que ambos sistemas son útiles, tienen sus desventajas.

- El Debugger requiere muchos recursos del sistema, haciendo incluso que el IDE vaya más lento.
- El debugger en ocasiones 'pierde la conexión' con el móvil emulado, y se comporta de forma extraña.
- El Logcat puede llenarte de SPAM la consola rápidamente si no tienes cuidado.

Dado que estás aprendiendo las bases, te recomiendo que aprendas ambas herramientas. Te van a ser útiles para cuando hagas Apps verdaderamente sofisticadas.

Importante – Debugear

Curiosamente, los programadores utilizan el debugger bastante poco. Según ganes experiencia te darás cuenta de que elaborar Apps sigue una lógica diferente a la que estás acostumbrado, y entenderás por qué. De todas formas, sigue siendo imprescindible en ciertos escenarios, como cuando quieres utilizar bases de datos ROM integradas en tu App.

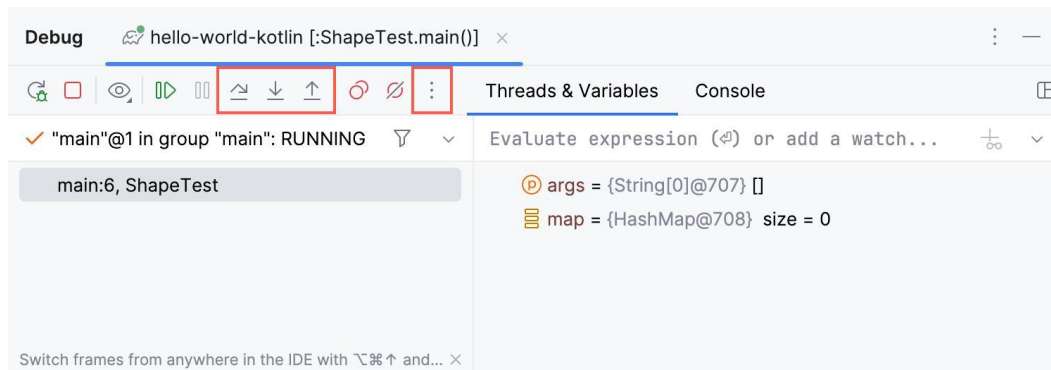
3.2 El Debugger

Para debugear una App, sigue los siguientes pasos:

Paso 1 – Breakpoint: Añade un punto de parada (Breakpoint) en la línea de código que deseas inspeccionar.

Paso 2 – Modo Debug: Ejecuta la App en el modo Debug. Se arrancará el emulador y la App se lanzará, deteniéndose en el punto de parada que has seleccionado. Nótese que se va a detener únicamente el hilo activo. El resto de los hilos de la App (de haberlos) puede que sigan su proceso de ejecución sin detenerse.

Paso 3 – Interfaz: Dispones de un interfaz con controles para debugear la App. Marcados en rojo tienes los controles de step-in, step-out, etc.



Fíjate que a la derecha te aparecen las **variables** que hay en ese momento en memoria. Finalmente, es posible que añadas una expresión que pueda ser **evaluada** en caliente en ese punto de parada.

Paso 4 – Modificar Valores: Android Studio permite que cambies al vuelo los valores de las variables mientras el debugger está parado. Esto permite simular errores y adelantarte el trabajo.

3.3 El Logcat

Es realmente sencillo lanzar una traza de mediante el Logcat. Basta con añadir en tu código algo como:

```
Log.d("MainActivity", "Usuario cargado correctamente")
Log.e("Login", "Error al autenticar")
```

Estas trazas saldrán por la ventana correspondientes del Logcat en tu Android Studio. Los niveles del log son los siguientes:

- ✓ **Log.v** → Verbose (ruido, casi nunca útil en producción)
- ✓ **Log.d** → Debug (desarrollo)
- ✓ **Log.i** → Information (eventos relevantes)
- ✓ **Log.w** → Warning (algo raro)
- ✓ **Log.e** → Error (fallo real)

Con cada traza, añades siempre una **etiqueta**. Esta etiqueta es importante, dado que nos servirá para filtrar las trazas de la ventana del Logcat en el Android Studio. También podremos filtrar por niveles de log, etc.

Por último, **limpia siempre tu App** de trazas de log al terminar de desarrollarla. Tanto por seguridad como por rendimiento.

Otra forma más elaborada podría ser utilizar este modelo para escribir trazas:

```
if (BuildConfig.DEBUG) {  
    Log.d("MainActivity", "Mensaje de debug")  
}
```

Esta variable puede ser puesta a false en cualquier momento en tiempo de compilación mediante el Gradle. Usualmente se hace durante el release de la App. Veremos más de esto próximamente.

```
android {  
    ...  
    buildTypes {  
        debug {  
            // Configuración para debug  
            minifyEnabled false  
            applicationIdSuffix ".debug"  
            versionNameSuffix "-DEBUG"  
        }  
        release {  
            // Configuración para release  
            minifyEnabled true  
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'  
            // Aquí BuildConfig.DEBUG será false automáticamente  
        }  
    }  
}
```